

Episodio 4: Verificación, validación y pruebas

Fuente para el podcast de repaso del parcial de **Aprobación Directa** de Ingeniería y Calidad de Software (opción múltiple, a libro abierto en papel). Contiene los capítulos 16, 17, 18 y 19 del resumen de estudio `resumen-ad.md`, con su numeración original: las referencias a otros capítulos remiten a ese resumen.

CONTENIDO

- | | |
|--|--|
| 16 Verificación y validación | 18 Las técnicas dinámicas |
| 17 La organización de las pruebas | 19 Las técnicas estáticas: revisiones |

16. Verificación y validación

Un sistema puede fallar de dos maneras muy distintas. Puede **no hacer lo que dice su especificación**: el recargo está mal calculado, una pantalla acepta un dato que debería rechazar. O puede hacer **exactamente lo que dice su especificación y aun así no servirle al cliente**, porque la especificación quedó mal relevada. Son dos problemas diferentes, se detectan con preguntas diferentes, y CMMI les dedica dos áreas de proceso de **nivel 3**.

16.1 La distinción

Es la pareja de conceptos más importante de la unidad y la más fácil de confundir, porque ambos son procesos de evaluación de productos, se ejecutan frecuentemente **de forma concurrente** y pueden compartir parte del entorno.

	VERIFICACIÓN (VER)	VALIDACIÓN (VAL)
Qué asegura	Que los productos de trabajo seleccionados cumplen sus requerimientos especificados	Que el producto se ajusta a su uso previsto cuando se sitúa en su entorno previsto
La pregunta	¿Estoy construyendo correctamente el producto?	¿Estoy construyendo el producto correcto ?
Contra qué contrasta	La especificación	La necesidad del usuario
Quién interviene	El equipo: es una cuestión interna	El cliente o usuario
Técnicas típicas	Revisiones entre pares , inspecciones, pruebas contra la especificación	Revisión de requisitos con el cliente , demostración de prototipos, pruebas de aceptación , piloto

ATENCIÓN El criterio que resuelve cualquier caso es **contra qué se contrasta y quién participa**, no qué tan terminado está el producto ni en qué fase ocurre. Unas pruebas de sistema ejecutadas por el equipo de QA sobre el producto terminado son **verificación**, porque se contrastan contra la especificación. Unas pruebas de aceptación con clientes reales son **validación**. Y revisar los requisitos **con el cliente** también es validación, aunque no se ejecute nada.

16.2 El criterio rápido para un enunciado

En el parcial casi nunca piden la definición: describen una actividad y preguntan el área. La regla tiene tres salidas. Si aparece el **cliente o usuario**, o se prueba el **uso real**, es **VAL**. Si se controla un **artefacto contra otro artefacto del proyecto** —casos de uso contra minutas, etiquetas contra el glosario— sin el cliente, es **VER**. Y si se controla contra un **estándar o procedimiento de la organización**, es **PPQA** (capítulo 21). Un ejemplo de clase, sobre un mismo caso de uso: "no se usó la etiqueta de tiempo que pide el estándar" es **PPQA**; "se usó *durante* cuando correspondía *reemplaza*" es **VER**, porque el contenido no satisface lo especificado.

El AD de 2025 lo preguntó con **diagramas de conjuntos**: **S**, la especificación; **P**, el producto implementado; **Test**, las pruebas ejecutadas. El planteo con Test casi entero **dentro de S** es **VER**: los casos salen de la especificación y miran si el producto la cumple. El planteo con Test casi entero **dentro de P** es **VAL**: se prueba el producto real en su entorno de uso, incluso comportamiento que la especificación no contempla.

Queda una frontera con **RD SP 3.5 Validar los requerimientos** (capítulo 14), que valida los requerimientos, temprano y sobre prototipos. **VAL** valida el **producto o sus componentes**: elegir los casos de uso que entran en la prueba de aceptación, o probar con el cliente en un piloto, es **VAL**.

ATENCIÓN Cuatro trampas que se repiten. "**Dinámica = validación**" es falso: las pruebas de integración y de sistema son dinámicas y son **VER**. **Una revisión entre pares es siempre VER**; una revisión **con el cliente** es **VAL**. **La inspección es una técnica, no un área**: la usa **VER** y también **PPQA**. Y aunque CMMI nombre la prueba de aceptación entre los métodos de **VER**, en el examen se tomó **aceptación** → **VAL**: la ejecuta el cliente para confirmar que el producto le sirve.

16.3 Metas y prácticas

VERIFICACIÓN (VER)

SG1 Preparar la verificación — SP1.1 Seleccionar los productos de trabajo a verificar · SP1.2 Establecer el entorno · SP1.3 Establecer procedimientos y criterios

SG2 Realizar revisiones entre pares — SP2.1 Preparar · SP2.2 Llevar a cabo · SP2.3 Analizar los datos

SG3 Verificar los productos seleccionados — SP3.1 Realizar la verificación · SP3.2 Analizar los resultados

VALIDACIÓN (VAL)

SG1 Preparar la validación — SP1.1 Seleccionar los productos a validar · SP1.2 Establecer el entorno · SP1.3 Establecer procedimientos y criterios

(VAL no tiene esta meta)

SG2 Validar el producto o los componentes — SP2.1 Realizar la validación · SP2.2 Analizar los resultados

ATENCIÓN Las **revisiones entre pares existen sólo en VER**. Si el enunciado describe una revisión entre pares y la opción dice **VAL/SP 2.1 Preparar las revisiones entre pares**, la respuesta es **NINGUNA**: hay que leer la sígla, no sólo el nombre de la práctica.

Para elegir la práctica sirve el **tiempo verbal**. En futuro —"será chequeada", "se distribuirá"— se está **preparando** (SG 1, o SP 2.1 si es una revisión entre pares); en presente —"está chequeando"— se está **realizando**; y "ya se documentaron los hallazgos, ahora se almacenan" es **analizar los datos**.

LO QUE DESCRIBE EL ENUNCIADO

PRÁCTICA

Decidir qué productos se verifican y con qué método ("se controlarán mediante inspección")

VER SP 1.1

Armaz el entorno: cargar la base de datos de pruebas

VER SP 1.2

Definir el tipo de revisión, el calendario, los roles; **distribuir** el producto a revisar

VER SP 2.1

LO QUE DESCRIBE EL ENUNCIADO	PRÁCTICA
Los pares chequean el producto y documentan los defectos	VER SP 2.2
Almacenar y proteger los datos de la revisión, para que no se usen para evaluar a las personas	VER SP 2.3
Controlar un producto contra sus requerimientos y registrar los resultados	VER SP 3.1
Elegir qué casos de uso entran en la prueba de aceptación	VAL SP 1.1
Acordar con el cliente los criterios de aceptación	VAL SP 1.3
Controlar con el usuario si las etiquetas de pantalla se entienden	VAL SP 2.1

16.4 Qué se valida y cómo

La validación se aplica a los productos de trabajo —requerimientos, diseños, prototipos— y al producto y sus componentes. Se hace **temprana e incrementalmente**, no al final.

El entorno debe **representar el entorno previsto**, y puede usarse completo o sólo en parte. Los métodos posibles son la discusión con usuarios, las demostraciones de prototipos, las demostraciones funcionales, los **pilotos** de materiales de formación, las pruebas realizadas por los usuarios finales y el análisis mediante simulaciones o modelado.

Es validable más de lo que se suele pensar: requerimientos y diseños, el producto y sus componentes, las **interfaces de usuario**, los **manuals de usuario**, los **materiales de formación** y la documentación del proceso.

La **verificación es incremental**: empieza por la verificación de los requerimientos, sigue con los productos de trabajo a medida que evolucionan, y culmina con la verificación del producto terminado.

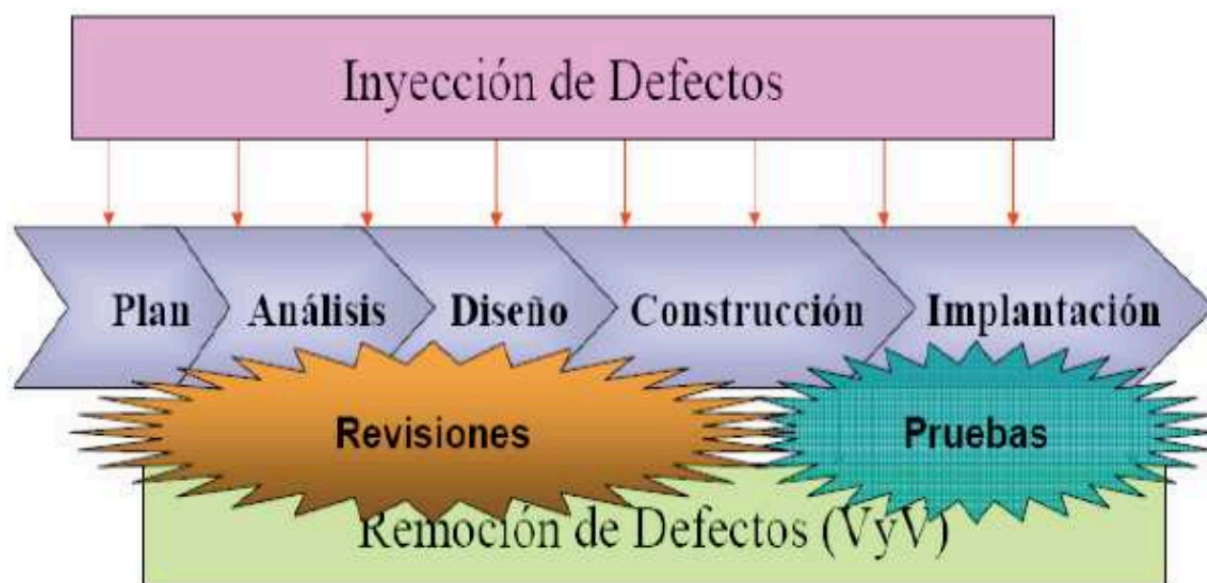
16.5 El grado de confianza

Verificar y validar no busca la ausencia total de defectos —que es inalcanzable— sino que el software sea **suficientemente bueno para su uso previsto**. Cuánta confianza hace falta depende de tres factores.

La **criticidad del sistema**: un sistema crítico exige confianza alta; un prototipo, mucho menos. Las **expectativas del usuario**, que vienen subiendo porque la tolerancia a fallos decrece. Y el **entorno de mercado**: con pocos competidores una empresa puede lanzar antes de estar completamente probada para llegar primero, y con un precio bajo los clientes toleran más defectos.

Conviene recordar que V&V son **procesos costosos**: en ciertos sistemas superan **la mitad del presupuesto total** de desarrollo. Por eso se planifican desde etapas tempranas.

16.6 Inyección y remoción de defectos



Los defectos **se inyectan en todas las fases** —plan, análisis, diseño, construcción e implantación—, no sólo al programar. Verificación y validación son el conjunto de actividades de **remoción**, y se reparten en dos franjas que se solapan: las **revisiones**, que son estáticas, cubren desde el plan hasta la construcción; las **pruebas**, que son dinámicas, cubren desde la construcción hasta la implantación.

De ahí la regla económica que ordena toda la unidad: **cuanto más tarde se detecta un defecto, más caro sale corregirlo**; una presentación de la cátedra calcula que en mantenimiento cuesta unas cien veces más que en requisitos. Por eso el riesgo de **relegar las pruebas al final** es detectar los defectos cuando corregirlos es más caro, y la respuesta es adelantar V&V: el modelo en V, la prueba en cada incremento, los prototipos y las revisiones desde los requisitos.

17. La organización de las pruebas

17.1 Qué es un caso de prueba

Caso de prueba: conjunto de **entradas, condiciones de ejecución y resultados esperados**, desarrollado para un objetivo o condición particular.

En su forma mínima se escribe como un par ordenado: (**valor de entrada** → **resultado esperado**). Sin resultado esperado no hay caso de prueba, porque no habría forma de saber si pasó o falló.

Nunca se prueba en producción: el entorno de pruebas debe estar **físicamente separado** y recrear las condiciones de producción. INTECO trata además las pruebas como un **subproyecto**, con su propio plan: en la planificación se define la estrategia, en el diseño se diseñan los casos, y en la construcción se ejecutan.

17.2 Los cuatro niveles de prueba

NIVEL	QUÉ PRUEBA	QUIÉN Y CÓMO	ÁREA
Unitarias	Cada módulo o componente aislado, antes de integrar	El propio desarrollador . Usa stubs y drivers para aislar el módulo	Las dos
Integración	La interacción entre módulos: defectos de interconexión	Se apoya sobre todo en el diseño	VER
Sistema	El comportamiento global contra la especificación, incluidos los no funcionales	Un equipo independiente , con caja negra y un entorno parecido a producción	VER
Aceptación	Que el producto satisface las necesidades del usuario	Un usuario o cliente	VAL

Ningún nivel reemplaza a otro. Que haya pruebas de integración no quita que se hagan las unitarias, y viceversa.

Para armar la integración hay tres estrategias. **Big-bang** ensambla todo de una vez: no requiere simular nada, pero consume mucho tiempo rastreando causas y descubre los problemas al final. **Bottom-up** avanza desde los módulos inferiores: necesita **drivers** en cada nivel y no encuentra problemas de diseño hasta muy avanzado. **Top-down** avanza desde los componentes superiores: necesita **stubs** que simulen los inferiores, pero **descubre rápidamente los errores de arquitectura**.

Las pruebas de aceptación **no buscan errores**: demuestran que el producto **está listo para producción**. Admiten tres modalidades: **alfa**, con un conjunto acotado de clientes preseleccionados en un entorno controlado; **beta**, con un conjunto más amplio; y **piloto**, con un conjunto reducido de departamentos del cliente y en **ambiente de producción**.

ATENCIÓN Unas pruebas que ejecuta **un equipo de la propia empresa sobre un prototipo** no son alfa ni beta, que requieren al cliente: son pruebas internas, y por área, **VER**.

17.3 Tipos de prueba

El **nivel** indica cuándo y sobre qué parte se prueba; el **tipo** indica con qué objetivo. Los tipos que se nombran en la materia son las pruebas funcionales, las de prestaciones, las de usabilidad, las de seguridad o acceso, las de configuración, las de instalación y carga inicial de datos, y las de migración de datos.

Dentro de las de **prestaciones** conviene distinguir cuatro que se confunden. Las de **carga** validan los requisitos de prestaciones definidos con escenarios realistas. Las de **capacidad** buscan el **punto umbral** a partir del cual las prestaciones se degradan, incrementando la carga hasta la saturación. Las de **estrés** estudian el comportamiento **en sobrecarga**, excediendo los límites, con foco en la integridad. Las de **estabilidad** miran el comportamiento **en el tiempo** bajo carga normal, para detectar mala liberación de recursos.

17.4 Regresión y confirmación

Son dos actividades distintas que se ejecutan juntas después de cada corrección.

La **confirmación** verifica que **el defecto corregido realmente se solucionó**: misma prueba, mismas condiciones, mismos datos. La **regresión** verifica que **el arreglo no rompió otra cosa**.

La regresión puede detectar tres situaciones: que el cambio **creó un error nuevo** (regresión local), que el cambio **reveló errores que ya existían** (de exposición), o que el cambio en un área **rompió otra área** del sistema

(remota). La estrategia más simple es la **fuerza bruta**, repetir todas las pruebas, y por eso la regresión es el mejor candidato a automatizarse.

17.5 Qué es probar

Myers sostiene que las pruebas pobres nacen de una **definición falsa** de qué es probar. Hay tres que suenan razonables y son trampas de opción múltiple: que la prueba sirve para **demostrar que no hay errores**, para demostrar que el programa **funciona correctamente**, o para **establecer la confianza** en que hace lo que debe. La correcta es otra:

Prueba (Myers): el proceso de **ejecutar un programa con la intención de encontrar errores**.

De ahí salen dos ideas que van contra la intuición: un **buen caso de prueba** es el que tiene alta probabilidad de encontrar un error todavía no descubierto, y un **caso exitoso** es el que **lo descubre**. Como las pruebas exhaustivas son imposibles, la economía de la prueba pasa a ser la consideración central. Dijkstra lo resumió: "*las pruebas sólo pueden demostrar la presencia de errores, no su ausencia*".

ATENCIÓN Un sistema que pasó todas las pruebas **no está libre de errores**: sólo se puede afirmar que **pasó los casos ejecutados** y alcanzó un grado de confianza para su uso previsto.

17.6 Los diez principios de Myers

Son el marco de actitud con el que se encara el testing, y condensan buena parte de lo anterior.

1. Una parte **necesaria** de un caso de prueba es la definición de la **salida prevista**.
2. Un desarrollador debe evitar **probar su propio programa**.
3. El personal de prueba **no debería depender** del área de desarrollo.
4. **Inspeccionar los resultados** de la prueba.
5. Escribir casos **tanto para las condiciones de entrada esperadas como para las no esperadas**.
6. Examinar un programa para comprobar **que no hace lo que no se supone** que haga.
7. Evitar casos de prueba **desechables y sin documentar**: hay que guardarlos para reejecutarlos.
8. **No planificar** el esfuerzo de pruebas suponiendo que no se encontrarán errores.
9. La probabilidad de encontrar errores adicionales en una sección es **proporcional al número de errores ya encontrados** en esa misma sección.
10. Las pruebas son una tarea **altamente creativa** y un desafío intelectual.

Los principios 2 y 3 merecen desarrollo, porque suelen preguntarse. Hay tres razones por las que quien desarrolla no debe probar su propio código. La primera es de actitud: **desarrollar es un proceso creativo y probar es un proceso destructivo**, y cuesta cambiar de una mentalidad a la otra sobre el trabajo propio. La segunda es la **visión de túnel**: quien desarrolló entiende su código de raíz y tiende a probar los caminos que ya sabe que funcionan. La tercera es la más grave: si el error está en **cómo se entendió el requerimiento**, quien lo entendió mal va a probar según su propio malentendido; por eso se prueba también **con el cliente**.

El principio 9 tiene una consecuencia práctica directa: conviene concentrar el esfuerzo de corrección **en los módulos donde más defectos aparecieron**, porque es donde es más probable que sigan apareciendo.

18. Las técnicas dinámicas

Las técnicas dinámicas ejecutan el código y buscan **fallos**. Existen porque **no se puede probar exhaustivamente**: las combinaciones de entradas posibles son inabarcables. Cada técnica es una forma sistemática de elegir un subconjunto de casos con alta probabilidad de encontrar defectos.

Antes de ver cada una, conviene tener el criterio de selección, porque los enunciados suelen describir la situación y pedir la técnica:

LO QUE DICE EL ENUNCIADO	TÉCNICA
Un campo con rango, longitud o formato; "valores válidos e inválidos"	Partición de equivalencia o valores límite
Varias condiciones que se combinan, reglas de negocio que se cruzan	Tablas de decisión
Una secuencia de estados, "pasa de X a Y"	Transición de estados
Flujo básico y flujos alternativos, escenarios de punta a punta	Casos de uso
Se dispone del código, cobertura de sentencias o decisiones	Caja blanca
Volver a probar lo que ya funcionaba después de un cambio	Regresión

18.1 Particionamiento de equivalencia

La idea es agrupar las condiciones de entrada que **el sistema trata igual**. Si el programa funciona para un valor de la partición, se asume que funciona para todos; si falla para uno, se asume que falla para todos. Por eso alcanza con probar **un representante por partición**.

Las particiones se dividen en **válidas** —las que el sistema debe aceptar— e **inválidas** —las que debe rechazar—. Las directrices habituales son: un rango como "10 a 100" da una partición válida —o **una por tramo**, si la regla de negocio cambia el resultado— y **dos inválidas** (por debajo y por encima); una lista como "ROJO, BLANCO, NEGRO" da una válida si todos se comportan igual, o **una por valor** si el comportamiento cambia, y una inválida; una condición de obligación como "letras mayúsculas" da una de cada una. El **vacío** va siempre como partición aparte. También existen las **particiones de salida**, que agrupan por resultado en vez de por entrada.

El formato con el que se trabaja en la materia tiene dos tablas encadenadas. La primera declara el **atributo**, su **dominio** y las particiones válidas e inválidas. Para un campo "día del mes" con dominio entero positivo entre 1 y 31, y un recargo que cambia cada diez días:

PARTICIONES	
Válidas	PV1) $1 \leq X \leq 10$ · PV2) $11 \leq X \leq 20$ · PV3) $21 \leq X \leq 31$
Inválidas	PI1) letras · PI2) $X \leq 0$ · PI3) $X > 31$ · PI4) vacío · PI5) imagen · PI6) carácter especial · PI7) cadena de caracteres

La segunda lista los casos de prueba, con un representante por partición y una fila por caso: **Caso | Partición | Entrada | Salida esperada**. Con las particiones de arriba serían diez casos: tres válidos (1, 12, 30) y siete inválidos, uno por cada forma de entrada rechazable.

ATENCIÓN Dos errores arruinan este ejercicio. El primero es **olvidar las particiones intermedias**: si un sistema se activa "por encima de 25" y se apaga "por debajo de 20", entre 20 y 25 hay una zona donde no pasa nada, y esa zona **es una partición válida**. El segundo es **dejar que las particiones se pisen**: si la válida llega hasta 31, la inválida empieza en 32, o sea $X > 31$, no $X \geq 31$.

18.2 Análisis de valor de frontera

Es una **mejora** del particionamiento, no una alternativa. En vez de un solo representante por partición, prueba **más de un caso en cada una**, concentrados en los **extremos**, porque es donde se agrupan los errores: si el código dice > 50000 en lugar de ≥ 50000 , un representante como 30.000 no lo detecta; 50.000,00 y 50.000,01 sí.

Para un rango de 10 a 100 se prueban **9, 10, 100 y 101**. Si se trabaja con dinero a dos decimales, el "siguiente valor" es 0,01. Para conjuntos ordenados, el primer y el último elemento. Hay valores especiales que conviene no olvidar: en minutos siempre 0 y 59; en fechas, los límites de mes y los **años bisiestos y no bisiestos**.

Para campos no numéricos el criterio cambia. Si el campo admite **una letra cualquiera**, alcanza un caso. Si admite un **conjunto cerrado** de N valores, va un caso por valor. Si es una **cadena de longitud fija**, se prueba con esa longitud exacta. Si acepta **hasta N caracteres**, van casos para las longitudes alrededor de N: N-1, N y N+1.

Cuando una consigna pide "cubrir todas las particiones válidas", pide **representantes**, no bordes. Cuando pide valores límite, pide los **extremos**. Es la diferencia entre las dos técnicas y es lo que se evalúa.

18.3 De las particiones a los casos de prueba

Los casos se escriben en uno de dos formatos. **Por campo**, atributo por atributo. **De función**, con una columna por atributo: se arranca con casos de **todos los valores válidos** y después va **un inválido por vez**, dejando el resto válido. En los finales se pidieron los dos formatos, así que hay que leer la consigna.

El AD de 2025 preguntó **qué hay que tener en cuenta al definir los casos de prueba a partir de las particiones**. Sí van las **entradas**, las **salidas esperadas** y el **comportamiento esperado** —es la definición misma de caso de prueba— y la **técnica a emplear**, porque partición pura y valores límite sacan valores distintos de la misma partición. No van **si la prueba es alfa o beta**, **si la ejecuta personal de desarrollo** ni las **limitaciones del lenguaje**, que clasifican pruebas pero no cambian el diseño del caso. La corrección sólo confirmó lo de la técnica y lo de alfa o beta; el resto se deduce de Myers y de INTECO.

Las resoluciones de finales que circulan como práctica repiten los mismos errores de traducción del enunciado:

EL ENUNCIADO DICE	LO CORRECTO
"Descuento si la edad es superior a 65 "	La partición empieza en 66
"Descuento si las noches son más de 6 "	Empieza en 7 ; los límites son 6 y 7
"Monto mayor a cero "	La inválida es ≤ 0 : el 0 también es inválido
"Los mayores de 17 ingresan pagando"	Partición ≥ 18 , no una partición de un solo valor
Un campo donde 0 significa "sin límite"	El 0 es válido , con partición propia
Una reunión con el cliente para evaluar la especificación	No es prueba de caja negra: es estática , y por el cliente, VAL

Y cuando el enunciado deja algo abierto, hay que **declarar el supuesto**: la cátedra corrige por la justificación.

18.4 Tablas de decisión

Se usan cuando **múltiples combinaciones de entradas** producen resultados distintos. A diferencia de las anteriores, que miran un campo por vez, esta técnica se centra en **la lógica y las reglas de negocio** que cruzan varios campos.

La tabla tiene filas de **condición** y filas de **acción**, y **cada columna es una regla de negocio**, es decir, un caso de prueba.

Armarla tiene cuatro pasos. Se identifican las **condiciones**, que son los atributos que deciden el resultado. Se identifican las **acciones**, que son los resultados distintos que el sistema puede ejecutar. Se generan las **columnas**, combinando los valores posibles de cada condición. Y se **reduce** la tabla, eliminando las combinaciones imposibles y colapsando las indiferentes.

Un ejemplo que aclara el conteo. Un gimnasio tiene socios de categoría Coulson, IronMan (10 % de descuento) o Hulk (15 %), y quien tiene la cuota al día recibe un 5 % adicional acumulable. Las condiciones son dos: la categoría y el estado de la cuota. Pero la categoría **no es binaria**: tiene tres valores mutuamente excluyentes. Entonces las columnas son $3 \times 2 = 6$, y las acciones son **cuatro**: aplicar 0 %, 10 %, 15 % y el 5 % adicional.

ATENCIÓN El error típico es contar toda condición como binaria y calcular 2^n columnas. Una condición puede tener tres o más valores, y en ese caso el producto cambia. Y a la inversa: cuando hay **condiciones incompatibles** —si el usuario no existe, la contraseña es irrelevante—, las combinaciones se colapsan y quedan menos casos de los que sugiere la potencia de dos.

18.5 Transición de estados

Se aplica a sistemas modelables como **máquina de estados finitos**, donde la salida ante la misma entrada **depende del estado anterior**. El ejemplo canónico es un trámite que pasa de Inscripto a Cursando y de ahí a Aprobado o Desaprobado.

Partiendo de la máquina de estados, la técnica permite revisar qué hace falta para llegar a cada estado y detectar incompatibilidades: **transiciones que faltan**, o estados de los que no se puede salir. Una prueba completa no se limita al camino feliz: debe incluir las **transiciones no válidas** —intentos fallidos, timeouts— y los **eventos no especificados**, como cancelar a mitad de camino.

18.6 Derivación de casos de prueba desde casos de uso

Ejercitan el sistema **de punta a punta**, siguiendo el recorrido real de un usuario. Por eso dan casos de **mejor calidad** que los armados campo por campo: validar una tarjeta en un cajero contempla muchas más cosas que probar tres números sueltos en un formulario.

El método tiene tres pasos. **Primero, los escenarios**: cada escenario es el **flujo básico** solo, o combinado con uno o más **alternativos**. En el cajero automático de la presentación de la cátedra, E1 es el retiro satisfactorio; E2, la tarjeta no válida; E3 y E4, el PIN incorrecto con y sin intentos restantes; y así con los fondos insuficientes y la cancelación.

Segundo, la matriz V/I: una fila por caso de prueba y una columna por cada condición o dato, incluidas las **condiciones que valida el sistema** aunque vengan de la base. **V** marca la condición válida, que sigue el flujo básico; **I**, la que **dispara el alternativo**; **n/a**, la que no llega a evaluarse.

CASO	ESCENARIO	TARJETA	PIN	FONDOS	RESULTADO ESPERADO
1	Retiro satisfactorio	V	V	V	Entrega el dinero
2	Tarjeta no válida	I	n/a	n/a	Expulsa la tarjeta con un mensaje
3	PIN incorrecto, sin intentos	V	I	n/a	Retiene la tarjeta
4	Fondos insuficientes	V	V	I	Mensaje; vuelve a pedir el importe

Tercero, se completa con valores reales. El primer caso es positivo del flujo básico; los demás son negativos del flujo básico y positivos de su alternativo, y un escenario puede necesitar más de un caso. Después se suman las **especificaciones complementarias**: rendimiento, seguridad, configuración e instalación.

ATENCIÓN El error típico está en la matriz. Para provocar "no hay vehículos disponibles", la resolución de un final marcaba como inválidas la fecha y la duración, que eran datos válidos: lo que dispara el alternativo es la **disponibilidad**, una condición que valida el sistema y que necesita su propia columna.

18.7 Caja negra y caja blanca

La distinción es qué información tiene quien prueba.

En **caja negra** sólo se ven entradas y salidas. Es rápida y no requiere acceso al código, pero cuando algo falla **no se sabe dónde está el error**. Todas las técnicas anteriores son de caja negra.

En **caja blanca** se dispone del **código fuente**, lo que permite saber cuántas sentencias y condiciones se ejecutaron y sacar un porcentaje de cobertura. Trabaja sobre tres estructuras: secuencia, selección e iteración.

TÉCNICA	OBJETIVO
Pruebas de sentencia	Ejecutar cada sentencia ejecutable al menos una vez
Pruebas de decisión	Evaluar cada decisión (IF-THEN-ELSE, DO-WHILE) en verdadero y falso
Pruebas de caminos	Recorrer cada camino de ejecución independiente. No prueba todas las combinaciones: con bucles serían infinitas

Conviene notar que un defecto puede manifestarse **aunque todas las sentencias se hayan ejecutado al menos una vez**, porque el problema aparece recién al **combinarse** ciertos caminos. La cobertura del 100 % de sentencias no garantiza ausencia de defectos.

Ver el código también **mejora los casos de caja negra**. Un final lo muestra con una función que devuelve la proporción A/B entre dos enteros: con el código a la vista —`return A/B`, sin ninguna validación— aparece un caso que hay que agregar, **B = 0**. En ese momento la prueba pasó de caja negra a caja blanca.

18.8 Técnicas basadas en la experiencia

Se usan cuando **no hay una especificación adecuada** o **no hay tiempo**. La **adivinación de errores** complementa a las técnicas formales y depende de la habilidad e intuición del técnico. Las **pruebas exploratorias** consisten en recorrer el software para entender qué hace, qué no hace y dónde está débil, diseñando las pruebas mientras se ejecutan.

19. Las técnicas estáticas: revisiones

19.1 Por qué existen

Las técnicas dinámicas necesitan el sistema andando, y eso limita cuándo se pueden aplicar. Las **estáticas** analizan documentos —requisitos, diseños, código, historias de usuario— **sin ejecutar nada**, y por eso se pueden aplicar en **cualquier momento** del ciclo de vida. Buscan **defectos**, no fallos.

Son la **primera forma de prueba aplicable** en un proyecto, y están vinculadas principalmente a la **verificación**.

	ESTÁTICAS (REVISIONES)	DINÁMICAS (PRUEBAS)
Qué encuentran	Defectos	Fallos
Qué necesitan	Documentos	El sistema ejecutable
Cuándo se aplican	En cualquier momento	Sólo con el producto andando
Ventaja	Se sabe dónde está el problema; se encuentran varios a la vez; se corrige temprano	Más rápidas de ejecutar
Desventaja	Más lentas	No se sabe dónde está el error

Son **complementarias**: ninguna reemplaza a la otra.

19.2 Beneficios y costo

Las revisiones mejoran la calidad y la comprensión de los entregables, validan que soportan la solución final, gestionan las expectativas del negocio, identifican tareas de alto riesgo y forman al equipo. Al reducir los errores que llegan a la etapa de pruebas, **acortan los períodos de prueba y bajan sus costos**.

Aun así, muchas organizaciones no las implementan, y la explicación que da la materia es que tienden a **sobreestimar su costo y subestimar sus beneficios**.

19.3 Formalidad

Una revisión puede ser informal o formal, y la diferencia está en si hay proceso.

Las **informales** no tienen proceso definido, no tienen roles y habitualmente no se planean. Cualquier intercambio entre pares cuenta: preguntarle a un compañero si le parece bien un pedazo de código es una revisión informal.

Las **formales** tienen objetivos definidos, proceso documentado, roles asignados a personas entrenadas, checklists y reglas, **reporte de resultados** y recolección de datos para el control del proceso.

La formalidad importa porque deja **trazabilidad documentada** de las acciones y decisiones, lo que permite demostrar después que los procedimientos se cumplieron.

19.4 El proceso y los tipos

El proceso básico es común a todas: se identifican los entregables a revisar, se arma la lista de participantes, los revisores **estudian** el documento por su cuenta, identifican problemas y se los **comunican al autor**, y el autor **responde y actualiza**.

TIPO	QUIÉN LO DIRIGE	FOCO	FORMALIDAD
Revisión informal	—	Encontrar defectos; documentar es opcional	Mínima
Walkthrough	El propio autor	Entendimiento común, evaluar contenidos, discutir alternativas	Media
Revisión técnica	Un moderador capacitado o experto técnico	Consenso técnico , con revisores expertos; no es búsqueda de defectos	Variable
Revisión entre pares	Colegas del mismo proyecto	Identificar y eliminar defectos temprano , de forma incremental	Media
Inspección	Un moderador formado, nunca el autor	Registrar defectos eficientemente: las discusiones se posponen. Seguimiento formal con criterios de salida	Máxima

19.5 Los cinco roles y los factores de éxito

El **moderador** dirige el proceso, determina junto con el autor el tipo de revisión y la composición del equipo, y hace el seguimiento. El **autor** creó el documento y busca mejorar su calidad. El **documentador** anota cada defecto y sugerencia. El **revisor** valida el material buscando defectos **antes** de la reunión. El **supervisor** decide destinar tiempo del proyecto a las revisiones y determina si se cumplieron los objetivos.

Para que una revisión funcione hacen falta un **objetivo claro y acordado**; elegir los **documentos más críticos**, como los requisitos o la arquitectura, **sin inspeccionar todo**; reservar las horas **en el plan del proyecto**; **formar** a los participantes; crear un **entorno seguro y no amenazante, enfocado en el producto y no en la persona**; y **documentar los defectos** con ubicación y descripción.

19.6 Análisis estático

Es la variante automatizada: busca defectos **sin ejecutar** el programa, pero **una vez escrito el código**, con herramientas llamadas analizadores estáticos.

Detecta variables **no inicializadas**, variables **no utilizadas**, **código inalcanzable**, inconsistencias entre módulos, vulnerabilidades de seguridad y violaciones de los estándares de programación. Encuentra además inconsistencias en los modelos, algo que las pruebas dinámicas no pueden hacer.

Entre las métricas de código que produce, la más usada es la **complejidad ciclomática**, que se calcula como el número de sentencias de decisión binarias más uno —o, sobre el grafo del código, aristas menos nodos más dos— y sirve para **estimar cuántas pruebas** necesita un componente.